# Stack Tracing, Simplified: The SFrame Story



## KERNEL RECIPES, 2025

Indu Bhagat, Linux Toolchain Team,
Oracle Canada

# Overview

- How it all started
- Where we are at now
- Looking ahead



# Problem With Existing Methods

## Frame Pointers
**[Simple but Unreliable]**

- Even `-fno-omit-frame-pointer -mno-omit-leaf-frame-pointer` may not be sufficient.
- 5-7% of stack traces may be unreliable [`Ref1`]

## EH Frame
**[Powerful but Complex]**

- Bulky stack tracer.
- Not suitable for stacktracing in resource constrained environments.



# The SFrame Solution: Bridging The Gap

SFrame was created to bridge the gap.  It provides a simplified representation of a subset of EH Frame format, offering the reliability of DWARF-based stack tracing, without the implementation complexity.

| FEATURE | FRAME POINTERS | SFRAME | EH FRAME |
|---|---|---|---|
| Reliability | Low | High | High |
| Stacktracer Implementation Simplicity | Low | High | Low |
| Asynchronous Stack Tracing | - | ~High | High |

~High: DRAP on x86_64, Signal Frame





# SFrame vs. EH Frame (Why not “EH Frame like” solution)

| | SFrame | EH Frame |
|---|---|---|
| Purpose | Stack tracing | Stack unwinding |
| Parsing Complexity/Data Retrieval | Stack offsets are encoded directly | Stack offsets are calculated by interpreting DWARF opcodes and their args |
| Encodings | 1byte, 2byte, 4byte | ULEB128, SLEB128,... |
| Number of tracked entities | CFA, FP, RA | CFA, callee-saved registers |

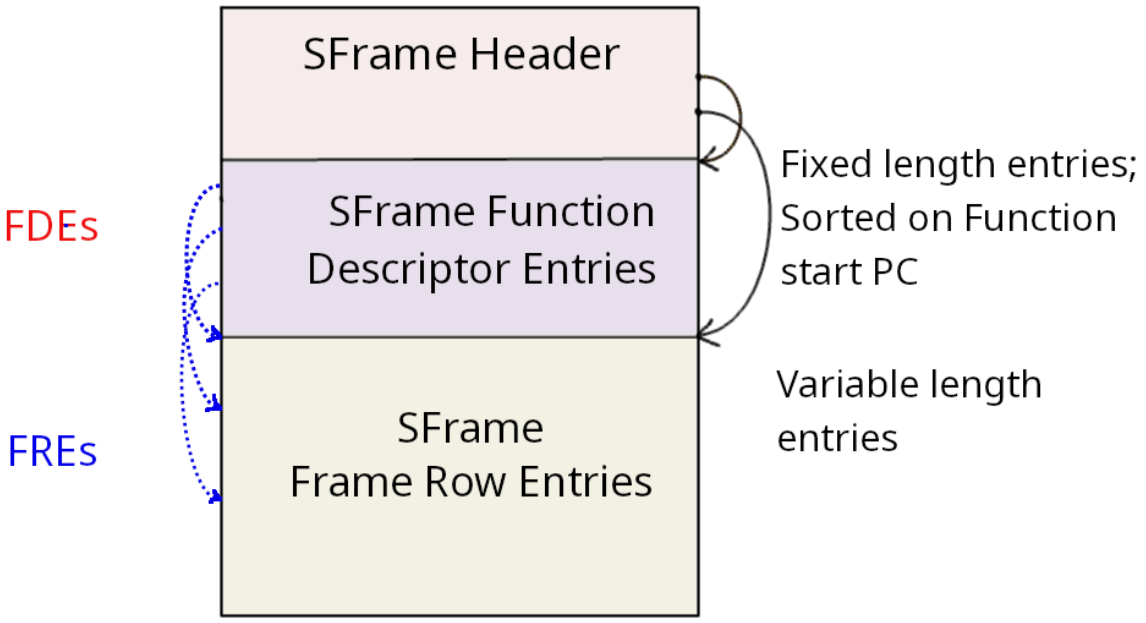
| | | |
|---|---|---|
| Function Index | Built-in | .eh_frame_hdr |
| Section Layout | Need to be merged | Concatenated |
| Section Type | SHT_GNU_SFRAME | SHT_PROGBITS |

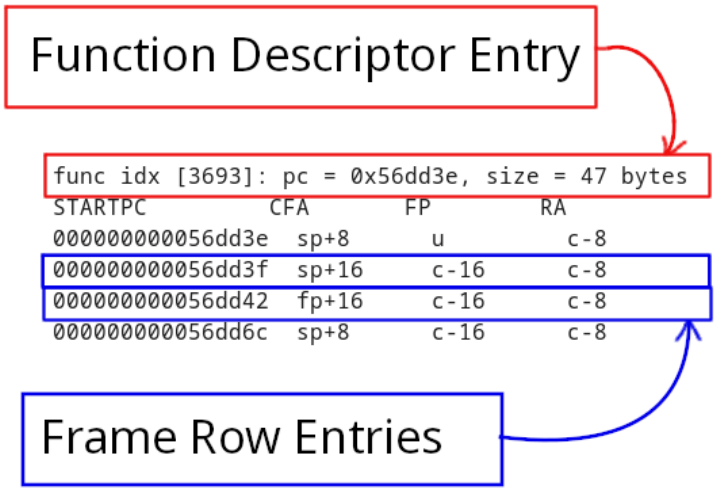


# Quick Details to SFrame



# Information layout in SFrame Section






# SFrame ELF section on GNU/Linux

- “.sframe” section.  Current version SFRAME V2 (Errata 1).
  - Section type SHT_GNU_SFRAME, Section flags SHF_ALLOC.
  - .sframe section sits in a read-only PT_LOAD segment.
- **[Access SFrame]** New program header type PT_GNU_SFRAME points to .sframe.
- **[Access FDE]** Flag SFRAME_F_FDE_SORTED in SFrame Header.
  - Index of (fixed length) FDEs sorted on start PC of function.
- **[Access FRE]** Use offset to access stack trace metadata for the desired PC range.



# SFrame Stack Trace Format (Recap)

- Minimal necessary information for stack tracing: CFA, FP, RA per PC
  - Fast stack tracing without the need to preserve frame pointer & where EH Frame stack tracing is not feasible
  - Stack offsets are encoded directly

| | | |
|---|---|---|
| GNU Binutils 2.40 | SFrame Version 1 (now obsolete) | Support for x86_64, AArch64 |
| GNU Binutils 2.41 | SFrame Version 2 | Fix for unaligned SFrame FDEs, Fix Rep Block representation |
| GNU Binutils 2.45 | SFrame Version 2 (Errata 1) | Relocatable links fixed, Support for s390x |
| GNU Binutils 2.46 (Dec 2025/Jan 2026) | SFrame Version 3 | Support for text > 2 GiB, Mark outermost frames, ... |





# SFrame Stack Trace Format (More Info)

- [SFrame: fast, low-overhead stack traces] https://lwn.net/Articles/940686/
- [SFrame SPEC] https://sourceware.org/binutils/docs/sframe-spec.pdf
- [SFrame WIKI]

  https://sourceware.org/binutils/wiki/sframe
- [SFrame V3 list]

  https://sourceware.org/binutils/wiki/sframe/sframev3todo

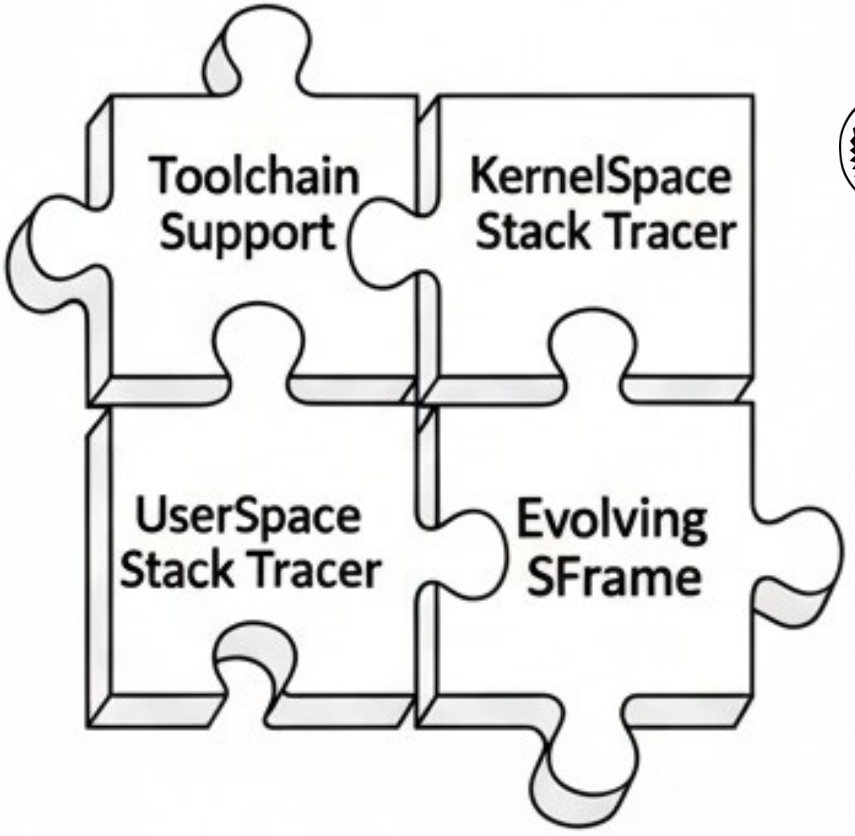




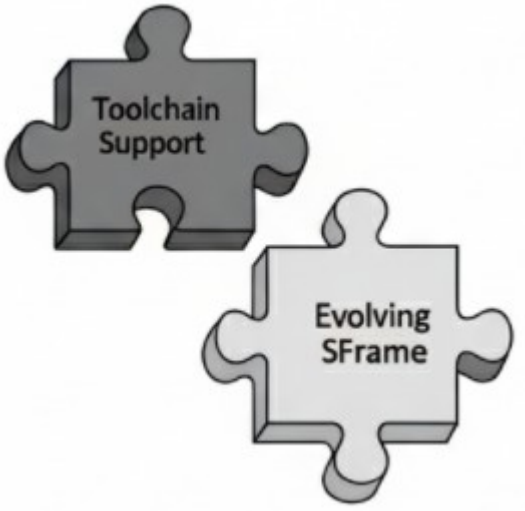
# Where are we at now...



# Exciting Community Involvement



# Toolchain Support – Key Highlights (s390x support in Binutils 2.45)




- s390x ("**flexible**" ABI) showed some fundamental conflicts with what SFrame requires:
  - RA/FP is either on stack or a fixed/single designated register.
  - Requires CFA tracking is SP or FP based.
- **SFrame will be the sole reliable method for s390x:** ( `-mbackchain` has issues.)
  - The topmost function might be still be in the function prologue.
  - In case of dynamic stack allocation, additional insns until backchain is setup.



# Toolchain Support – Key Highlights (s390x support - 2)

| Offset ID | Interpretation in AMD64 | Interpretation in AArch64 |
|---|---|---|
| 1 | CFA = BASE_REG + offset1 | CFA = BASE_REG + offset1 |
| 2 | FP = CFA + offset2 | RA = CFA + offset2 |
| | | FP = CFA + offset3 |

- **[Silver Lining]** SFrame FREs are blobs of data; SFrame allows these to be **treated in ABI-specific** ways.
  - E.g., offset = ((regno << 1) | 1) for RA in leaf functions.





# Toolchain Support – Key Highlights (s390x support - 3)

- Fine print in ABI specification adjusted.

```
commit f004218259791e141c0df738d32bc49f684570e8
Date:   Wed Sep 25 15:15:32 2024 +0200

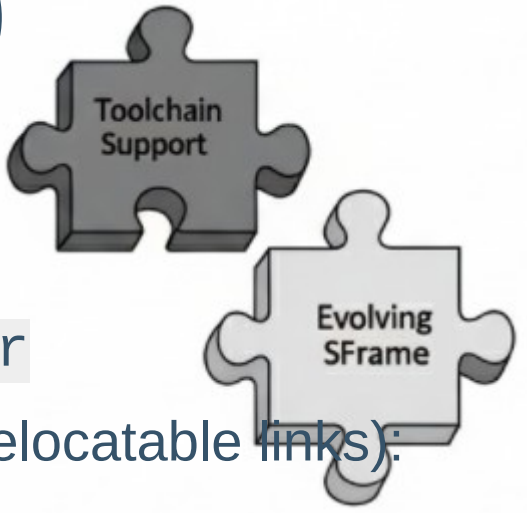
    Add information about the frame pointer register

    So far the frame pointer register was not documented.  Since it
is at least relevant to inline assemblies and for unwinding,
document it as another "register role" to be aware of.
```

- [Orthogonal] Making **design-time considerations** for fast stack tracing in the psABI/calling conventions/register roles can go a long way.



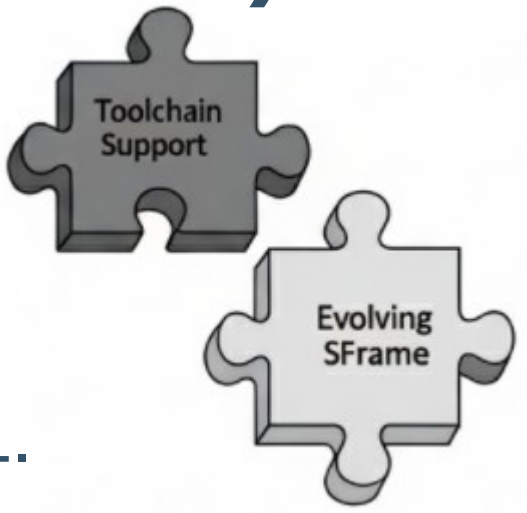
# Key Highlights from Binutils 2.45 (SFrame V2, Errata 1)




- Relocatable links (`ld -r` ) fixed
- What a nightmare of a bug/thinko!
  - Bug 32666 - Incorrect .rela.sframe when using `ld -r`
- There were two sets of encodings (and buggy SFrame relocatable links):
  - (#1) `ET_REL` generated by GNU `as` had `sfde_func_start_address` as offset from the field itself.
  - (#2) `ET_DYN, ET_EXEC,` and `ET_REL` (relocatable links) generated by ld had `sfde_func_start_address` as offset from start of section.



# Key Highlights from Binutils 2.45 (SFrame V2, Errata 1 Continued)




- Avoid misusing R_*_PCREL32 relocs.
  - SFrame desires R_*_SECREL.
- Introduce a new flag SFRAME_F_FDE_FUNC_START_PCREL.
  - SFrame FDE func start addr is an offset from field itself.



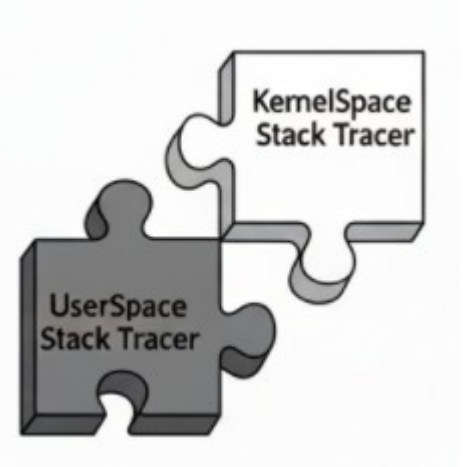
# Key Highlights (Upcoming Binutils 2.46)




- Evolving SFrame.
  - Plan for SFrame V3 (Binutils 2.46).
- Distro-wide build testing
  - GNU `as` (master) has new configure time option: `--enable-default-sframe`
  - GNU `as` emits .sframe for all objects it assembles.
  - Warnings and errors are silenced.
- Gentoo (https://wiki.gentoo.org/wiki/Project:Toolchain/SFrame), Fedora (https://fedoraproject.org/wiki/Changes/SFrameInBinaries)



# Key Highlights – Userspace Stack Tracing

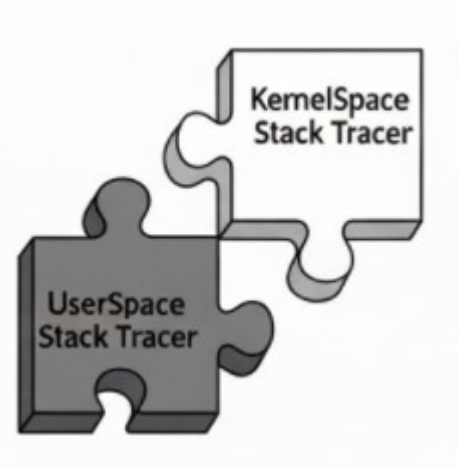



- SFrame aware backtrace () in glibc (glibc 2.42).
  - Experimental feature added to glibc 2.42.
  - Can be enabled using `--enable-sframe`.
  - Support for x86_64 and AArch64.
  - Minimum Binutils requirement is 2.45 (SFrame V2, Errata 1).
- Automatically employed whenever backtrace() function is invoked, if glibc is built with SFrame support.
  - Falls back on DWARF-based stack tracing.



# Key Highlights - Userspace Stack Tracing in Linux Kernel

- Userspace stack tracing in Linux kernel.
  - Deferred stack tracing infrastructure.
    “Deferred stack traces, how they work and the issues they have”, Tracing Summit 2025.
    https://cfp.tracingsummit.org/ts2025/talk/NZJNGS/
  - perf use of  deferred stack tracing infra.
  - SFrame stack tracer.
- Kernelspace stack tracing in Linux kernel.
  - ARM64 livepatching.






# Where are we headed ...



# Towards SFrame V3

- **[Robustness]** Support text > 2 GiB.
- **[Reliability]** Mark outermost frame.
- **[Performance]** Avoid unaligned accesses in SFrame FRE. **[WON'T FIX]**
- ...
- SFrame V3 TODO list and planning (https://sourceware.org/binutils/wiki/sframe/sframev3todo)



# Support for text > 2 GiB

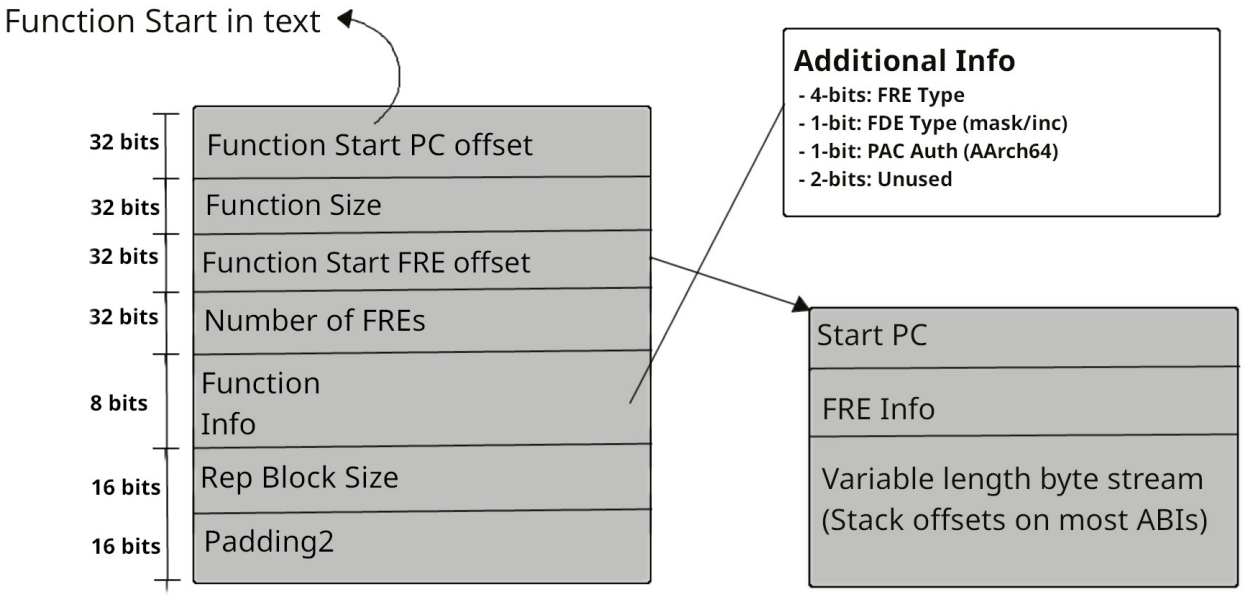
- Currently, GNU ld errors out if it sees the offset not representable as 32-bit signed int.

```
test.o:(.sframe+0x1c): relocation truncated to fit: R_X86_64_PC32 against `.text'
```

- May be limiting to some class of large applications (See previous discussion here and here).



# Support for text > 2 GiB




SFrame Version 2
Function Descriptor Entry



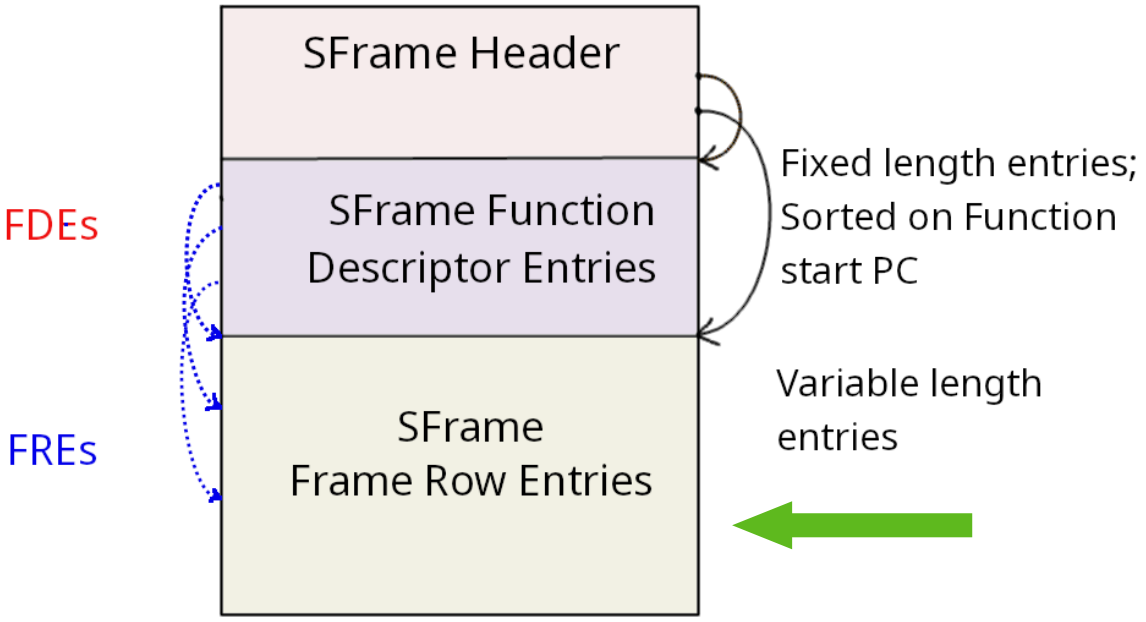


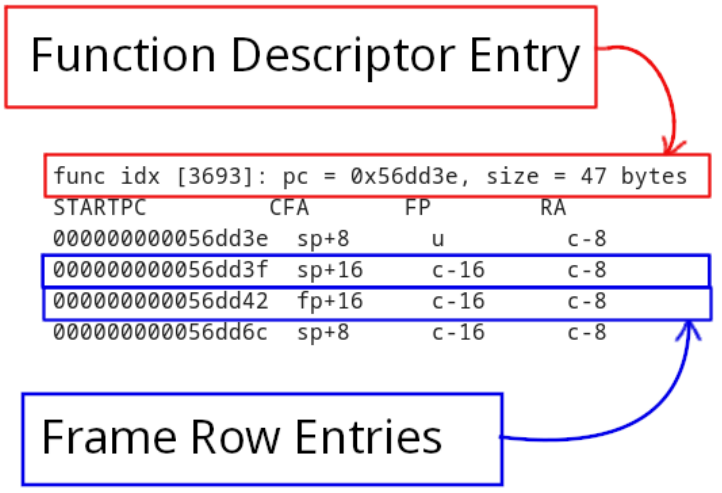
# Mark Outermost Frame

- Encode explicitly the outermost frame.
- DWARF CFI annotation.
  - .cfi_undefined RA (DW_CFA_undefined RA).
- SFrame will represent this with an FRE with no offsets.
- Helps stack tracers disambiguate between truncated stack traces from crisp/complete stack traces.



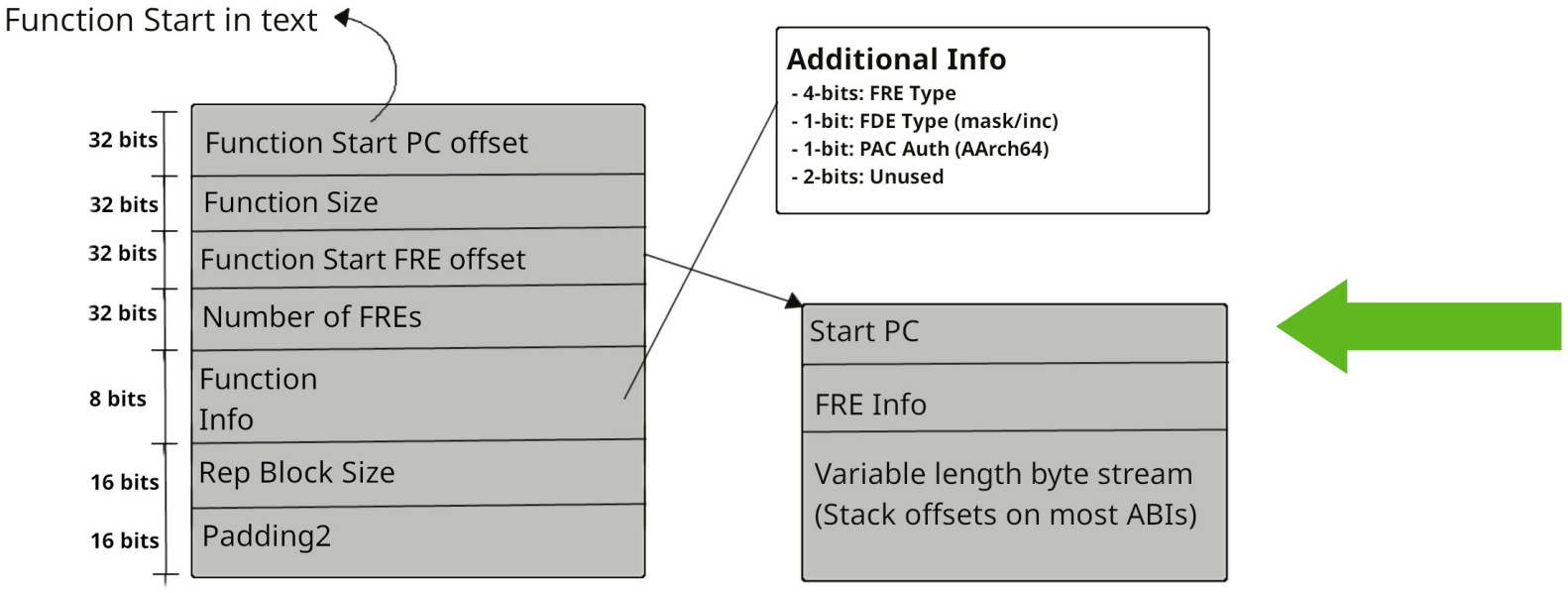
# Unaligned Accesses in SFrame FRE






# Unaligned Accesses in SFrame FRE

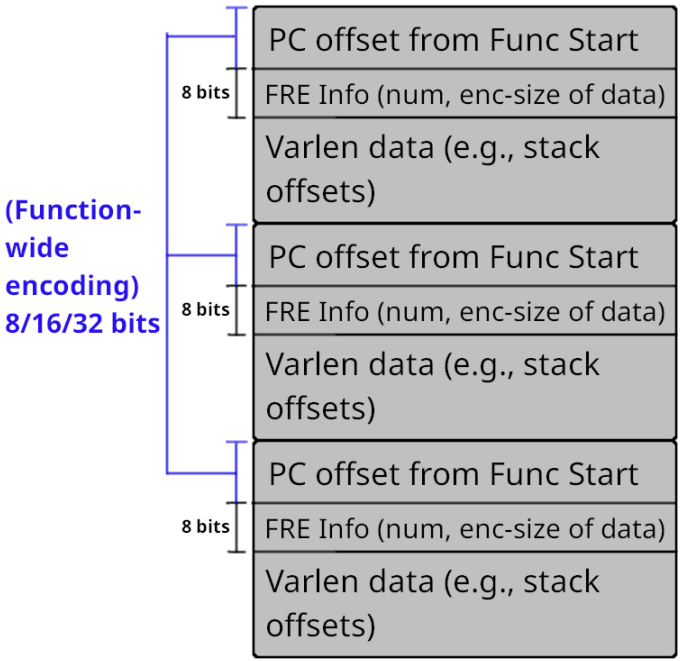



SFrame Version 2
Function Descriptor Entry





# Unaligned Accesses in SFrame FRE




- All FREs of a function occupy contiguous space.
- Access to Start PC offset, stack offsets may be unaligned.



# Forsaking Compactness for minor Performance gains

**NOT RECOMMENDED**

```
/* Approach 1a.  */
struct sframe_frame_row_entries {
        u8|u16|u32 start_addrs[N];
        u8 sfre_infos[N];

        s8 offsets8[M8];
        s16 offsets16[M16] __aligned(2);
        s32 offsets32[M32] __aligned(4);
};
```

```
/* Approach 1b.  */
struct sframe_frame_row_entries {
        u8|u16|u32 start_addrs[N];
        u8 sfre_infos[N];
        u16 offset_index[N]; /* Offset fr

        s8 offsets8[M8];
        s16 offsets16[M16] __aligned(2);
        s32 offsets32[M32] __aligned(4);
};
```

**Discussion: here**



# Trade-off Compactness for performance

- The central challenge is that any alternative must demonstrate a clear performance improvement while avoiding significant size overhead. Introducing "bloat" to the format to solve a potential performance issue is considered a poor trade-off.
- In future, if an architecture does not support unaligned accesses, decoding routines will need to use memcpy for the 16-bit/32-bit data reads.



# Summary

- SFrame, the “Simple Frame” stack trace format.
- Iterations to fit the role of user space stack tracing format for multiple architectures.
- Active community involvement in multiple pieces.



# Looking Forward To

- SFrame support in other toolchains.
- SFrame-based user space stack tracing in the Linux kernel.
- Distribution-wide enablement.
- Consolidated stack tracer implementations.
  - Fast, reliable stacktraces!



# Feedback/Comment/Participate

binutils@sourceware.org

linux-toolchains@vger.kernel.org

IRC channel: #sframe at irc.oftc.net

# Thanks!



# Extra Slides





# Deferred Userspace Stack Tracing

- **Problem**: Collecting userspace stack traces (using SFrame) from the kernel may require paging in memory, which is not allowed in certain contexts (e.g., NMI context).
- **Solution**: Defer the stack trace collection.
  - **Request**: The kernel requests a userspace stack trace and stores a "cookie" or token to identify it.
  - **Delay**: The actual stack trace collection is postponed until the task is about to return to userspace, a point where it's safe to handle page faults.
  - **Collect & Correlate**: The stack trace is collected and then matched with the corresponding kernel context using the saved cookie.
- “Deferred stack traces, how they work and the issues they have”, Tracing Summit 2025
  https://cfp.tracingsummit.org/ts2025/talk/NZJNGS/

